IEEE Access
Multidisciplinary | Rapid Review | Open Access Journal

Date of publication xxxx 00, 0000, date of current version xxxx 00, 0000.

Digital Object Identifier 10.1109/ACCESS.XXXX.XXXXXXX

Efficient Private Transformer Inference With GPU-Accelerated Secure Three-Party Computation

SHUSEN LIN[1]

[1][TODO: Yancheng Teacher college, Nanjing Tech University, Yancheng, China(email: bzsn20020212@njtech.edu.cn)Department, Institution, City, Country (e-mail: author@example.com)]

Corresponding author: [TODO: Name and e-mail address].

[TODO: Funding statement.]

ABSTRACT Transformer inference over sensitive inputs and proprietary model parameters requires cryptographic protection without sacrificing the model structure that provides its utility. Secure multiparty computation offers a natural protection boundary, but Transformer workloads combine communication-intensive nonlinear functions with large linear layers whose execution characteristics differ sharply from those of convolutional networks. This article develops a GPU-accelerated secure three-party computation design for private Transformer inference on top of the hpmpc and PIGEON software stack. The design assigns dense matrix operations to a CUDA path and expresses nonlinear layers through mixed arithmetic and Boolean secret-sharing primitives. Its implemented attention Softmax protocol uses a secure row-wise maximum, range-reduced polynomial exponentiation, local reduction over arithmetic shares, and a lookup-seeded Newton–Raphson reciprocal. We further specify integration contracts for secure LayerNorm and a feed-forward-network-plus-LayerNorm pipeline so that the remaining Transformer operators can share the same fixed-point representation and execution interface. A controlled validation plan separates operator-level numerical error from end-to-end system performance and records latency, communication, accelerator utilization, memory consumption, and model quality under identical inputs and parameters. [TODO: Insert the final quantitative results only after controlled three-party experiments.]

INDEX TERMS GPU acceleration, privacy-preserving machine learning, secure multiparty computation, secure three-party computation, Transformer inference.

I. INTRODUCTION

TRANSFORMER models have become a general computational substrate for language understanding and generation. Their central operation, self-attention, relates each token to other tokens in a sequence, while position-wise feed-forward networks (FFNs) transform the resulting representations [1]. BERT demonstrated that a deep bidirectional Transformer encoder can be pretrained once and adapted to a broad range of language tasks [2]. In many deployment settings, however, the input may contain medical, financial, enterprise, or personal information, and the model parameters may themselves be a valuable service-provider asset. Sending either side in plaintext creates a privacy boundary that is unacceptable for sensitive inference.

Secure multiparty computation (MPC) permits several parties to evaluate a function over secret-shared values without revealing the underlying inputs. Private inference can therefore protect both the client's query and the model owner's parameters, subject to the stated threat model. Early secure neural-network systems established practical hybrid protocols for convolutional networks by combining efficient linear algebra with specialized mechanisms for nonlinear activations [3], [4]. Mixed-protocol frameworks further showed that arithmetic, Boolean, and garbled representations can be selected according to the operation being evaluated [5]. These ideas are foundational, but they do not by themselves remove the distinctive costs of a Transformer.

The difficulty is not simply that Transformers are large. Their linear and nonlinear components stress a secure system in different ways. Matrix multiplications in attention and FFN blocks offer abundant data parallelism and map naturally to GPUs. By contrast, Softmax, LayerNorm, comparisons, fixed-point truncation, and reciprocal or inverse-square-root evaluation require interactive secure primitives. Softmax is

especially important because every attention head normalizes an entire score row rather than a small classification vector. A design that accelerates only matrix multiplication can leave communication-heavy nonlinear layers on the critical path; a design that optimizes only approximation degree can underutilize the available GPU.

Recent private-Transformer systems have addressed parts of this problem. Iron supplied dedicated protocols for Transformer operators in a two-party setting [6]. MPCFormer traded exact nonlinear evaluation for MPC-friendly approximations and recovered model quality through knowledge distillation [7]. PUMA designed secure Transformer functions, including Softmax and LayerNorm, with the goal of avoiding model retraining [8], while Ditto combined quantization-aware training with secure quantized inference [9]. In parallel, CryptGPU and Piranha demonstrated that GPUs can substantially broaden the practical design space of secret-sharing-based machine learning and MPC [10], [11]. Nevertheless, the interaction between Transformer-specific nonlinear protocols, high-throughput three-party execution, and GPU-backed linear layers remains an engineering problem that must be solved at the framework level.

This work studies that interaction in hpmpc and its PIGEON private-inference layer [12], [13]. We use Trio, the framework's semi-honest three-party protocol, and retain a single fixed-point representation across attention and FFN blocks. Dense products are assigned to the existing CUDA matrix-multiplication path, while nonlinear operators are decomposed into arithmetic-share operations and Boolean-path comparisons. The first implemented Transformer operator is a row-wise secure Softmax for BERT attention. It performs maximum subtraction through a comparison tree, evaluates a range-reduced polynomial exponential, reduces each row locally over arithmetic shares, and applies a lookup-seeded Newton–Raphson reciprocal.

The present data-independent draft makes three concrete contributions and draws a clear boundary around work still in progress:

- It defines a mixed-protocol execution architecture for mapping BERT-base attention and FFN blocks onto hpmpc/PIGEON with GPU-backed linear algebra and secure nonlinear operators.
- It implements an attention-oriented secure Softmax for score tensors of shape $[B, H, L, L]$, with a fixed-point numerical path designed for $L \leq 128$.
- It specifies framework-level contracts for LayerNorm and an FFN-plus-LayerNorm pipeline. Their protocol implementations and end-to-end performance are not claimed as completed in this draft and will be evaluated only after controlled three-party validation.

The remainder of the article introduces the MPC and Transformer background, positions the design relative to prior private-inference and GPU-MPC systems, and describes the operator protocols and framework integration. Experimental claims are intentionally deferred until identical inputs, model parameters, protocol settings, and plaintext references have been used across all evaluated configurations.

II. BACKGROUND AND MOTIVATION

A. THREE-PARTY SECURE COMPUTATION

Let P_0, P_1, and P_2 denote three computing parties. A value x is represented by shares rather than by its plaintext value. We write $\langle x \rangle^A$ for an arithmetic sharing over the ring $\mathbb{Z}_{2^k}$ and $\langle x \rangle^B$ for a Boolean sharing. Arithmetic sharing makes addition and public-scalar multiplication local, whereas multiplication of two secret values requires a protocol interaction. Boolean sharing is useful for bit-level operations, comparisons, and sign extraction. A mixed-protocol execution converts between representations only when the next operator benefits from a different circuit domain [5].

Our implementation uses hpmpc's Trio configuration, identified by `PROTOCOL=5`. The framework documents Trio as a semi-honest three-party protocol with preprocessing support and the complete primitive set needed by PIGEON. We assume that parties follow the protocol but may attempt to infer additional information from their local views. The manuscript will state the precise corruption threshold and output-release policy after the final deployment configuration is frozen; no malicious-security claim is made.

Neural-network values are encoded in the ring by fixed-point scaling. For a real number x, the integer encoding is

$$\widetilde{x} = \lfloor x\, 2^f \rceil \bmod 2^k, \tag{1}$$

where $k = 32$ and $f = 20$ in the Transformer path. A product of two encoded values carries $2f$ fractional bits and must therefore be truncated. Truncation choice matters for both numerical error and MPC cost, and prior work has shown that fixed-point truncation should be treated as a first-class systems decision rather than an implementation detail [14]. The current Softmax path uses exact truncation (`TRUNC_APPROACH=2`) after secure fixed-point products.

B. TRANSFORMER INFERENCE

For a sequence representation $X \in \mathbb{R}^{L \times d}$, one attention head forms queries, keys, and values by learned projections and computes

$$\mathrm{Attention}(Q, K, V) = \mathrm{Softmax}\left(\frac{QK^{\mathsf{T}}}{\sqrt{d_k}} + M\right) V, \tag{2}$$

where M is an optional attention mask. Multi-head attention evaluates this expression for H heads, concatenates their outputs, and applies an output projection [1]. BERT-base uses repeated encoder blocks containing multi-head attention, residual connections, LayerNorm, and a two-layer FFN [2].

The linear projections and FFN matrix multiplications dominate the public operation count, but operation count alone is a poor predictor of secure latency. Additions over arithmetic shares are local; secure multiplication introduces communication; comparisons typically require Boolean-domain work; and transcendental functions must be replaced by secure approximations. Consequently, a compara-

tively small nonlinear layer can introduce many synchronized rounds and interrupt otherwise efficient GPU execution.

C. WHY ATTENTION SOFTMAX IS A SYSTEMS BOTTLENECK

For scores $x_1, \ldots, x_L$, Softmax returns

$$p_i = \frac{\exp(x_i - m)}{\sum_{j=1}^{L} \exp(x_j - m)}, \qquad m = \max_j x_j. \tag{3}$$

Subtracting the row maximum is numerically necessary but does not constrain every centered score to $[-\ln 2, 0]$; it guarantees only $x_i - m \leq 0$. A secure implementation must therefore combine maximum evaluation, tail handling, exponent approximation, summation, and division. For $L = 128$, each BERT-base layer evaluates this normalization for every query position and every attention head. Reusing a small-class CNN output routine is not sufficient: an argmax-oriented classification head neither returns normalized attention probabilities nor addresses the larger row dimension.

D. MOTIVATION FOR GPU–PROTOCOL CO-DESIGN

CryptGPU and Piranha show that secret-sharing workloads can benefit from GPU execution when protocol operations and memory movement are organized around accelerators [10], [11]. PIGEON, meanwhile, provides a PyTorch-like architecture interface and can use hpmpc protocols together with CUDA-backed general matrix multiplication [13]. This combination motivates a heterogeneous design: keep large regular products on the GPU path, retain communication-efficient local reductions in arithmetic sharing, and invoke the Boolean path only for comparisons and bit-dependent selection. The objective is not merely to offload more code to the GPU, but to minimize domain conversions, communication rounds, and avoidable tensor materialization across an entire Transformer block.

III. RELATED WORK

A. SECURE NEURAL-NETWORK INFERENCE

Gazelle combined homomorphic encryption for linear layers with garbled circuits for nonlinear layers, establishing a practical hybrid design for two-party neural-network inference [3]. Delphi paired secure protocols with an architecture-search component that selects network structures according to secure-inference cost [4]. ABY3 developed a mixed-protocol framework for three-party machine learning, supporting conversions among arithmetic, Boolean, and binary-circuit representations [5]. These systems established core principles that remain relevant to Transformer inference: linear and nonlinear operators should not be forced into a single cryptographic representation, and conversion costs must be included in system design.

Our setting differs in its focus on a high-throughput, semi-honest 3PC implementation for Transformer encoder inference. We build on hpmpc, which studies high-throughput honest-majority MPC across network settings, and PIGEON, which adds model import, neural-network layers, and protocol-independent inference to hpmpc [12], [13]. Rather than proposing a new general-purpose sharing scheme, this work adds Transformer-oriented operators and an accelerator-aware execution path to the existing stack.

B. PRIVATE TRANSFORMER INFERENCE

Iron was among the first systems to address private Transformer inference directly. It introduced protocols for matrix multiplication and the nonlinear Softmax, GELU, and LayerNorm functions in a two-party client–server design [6]. MPCFormer replaced costly nonlinearities with MPC-friendly approximations and used knowledge distillation to reduce the resulting quality loss [7]. PUMA developed secure protocols for Transformer functions, including embedding, Softmax, GELU, and LayerNorm, and emphasized inference without task-specific model retraining [8]. Ditto pursued a complementary direction by combining quantization-aware distillation, efficient representation conversion, and secure quantized operators [9].

These systems expose three broad design choices. First, a protocol may preserve the pretrained model and invest more work in high-quality secure approximations. Second, it may modify or retrain the model to reduce cryptographic cost. Third, it may reduce precision through quantization and design protocols around the resulting representation. Our design follows the first path for the initial BERT-base target: it retains the standard attention structure, uses a 32-bit fixed-point representation, and approximates nonlinear functions inside the secure protocol. This choice isolates systems and numerical effects from retraining effects, although future quantized variants remain possible.

C. GPU-ACCELERATED MPC

CryptGPU executes privacy-preserving machine-learning operations over secret-shared values with CUDA kernels and demonstrates the value of keeping secure tensors on the GPU [10]. Piranha provides a modular GPU platform for secret-sharing protocols and supports several party counts and security models through reusable protocol and kernel layers [11]. PIGEON incorporates CUDA matrix multiplication and convolution into a high-throughput inference framework [13]. Our work is closest to this line of systems research, but its unit of optimization is a Transformer block: GPU matrix multiplication must be coordinated with comparison-heavy Softmax, normalization, truncation, and residual paths rather than evaluated as an isolated accelerator kernel.

D. POSITIONING

The proposed system occupies the intersection of three lines of work: mixed-protocol 3PC, private Transformer operators, and GPU-oriented secure computation. Relative to prior private-Transformer systems, the intended contribution is an implementation and evaluation in the hpmpc/PIGEON/Trio stack with explicit accelerator and network measurements. Relative to general GPU-MPC systems, the intended contribution is a Transformer-specific operator pipeline. The empir-

[TODO: Replace with the final system diagram: secret sharing → GPU linear layers → mixed-protocol nonlinear layers → reconstruction.]

FIGURE 1. Planned overview of the GPU-accelerated three-party Transformer inference pipeline.

ical magnitude of any latency, throughput, communication, or utilization improvement is deliberately left open until the full BERT-base pipeline and controlled baselines are available.

IV. METHOD

A. SYSTEM AND THREAT MODEL

The deployment contains three non-collocated compute parties P_0, P_1, and P_2. Model parameters and input tensors are secret-shared before evaluation, and the parties execute the same compiled computation graph. The current target is a semi-honest 3PC deployment using Trio through hpmpc. The output recipient and reconstruction policy are configurable and must be fixed for each experiment. Network encryption, malicious behavior, side channels, denial of service, and leakage through intentionally released model outputs are outside the current protocol claim.

The software stack has three layers. hpmpc provides arithmetic and Boolean secret shares, communication, preprocessing, fixed-point primitives, and protocol selection. PIGEON/FlexNN provides neural-network architecture construction and model execution. Transformer functions are registered through a new function identifier and call the same share types as existing PIGEON layers. This arrangement avoids a parallel runtime and makes the new operators available to the framework's process vectorization, split-role execution, and CUDA general-matrix-multiplication path.

B. ACCELERATOR-AWARE EXECUTION ARCHITECTURE

Figure 1 will summarize the final data path. The architecture divides each Transformer block into four operator classes:

1) *linear projections and FFN products*, executed through the CUDA matrix-multiplication path;
2) *local affine operations and reductions*, evaluated directly over arithmetic shares without communication when algebraically possible;
3) *secure products and truncations*, evaluated in the arithmetic path with batched communication; and
4) *comparisons and bit-dependent selection*, evaluated through the Boolean path and converted back to arithmetic shares.

The compilation target uses 32-bit ring elements with 20 fractional bits. hpmpc packs multiple ring elements into a wider data type and can execute several independent processes. Tensor dimensions are therefore separated from packed execution lanes: a logical Softmax row always contains L attention scores, while batching and vectorization determine how many rows are evaluated concurrently. This distinction prevents process layout from changing the mathematical operator.

Algorithm 1 Secure row-wise attention Softmax

Require: Arithmetic shares $\langle x_1, \ldots, x_L \rangle^A$
Ensure: Arithmetic shares $\langle p_1, \ldots, p_L \rangle^A$
1: $\langle m \rangle^A \leftarrow \mathrm{TreeMax}(\langle x \rangle^A)$
2: $\langle \delta_i \rangle^A \leftarrow \langle x_i - m \rangle^A$
3: $\langle e_i \rangle^A \leftarrow \mathrm{NegExp}(\langle \delta_i \rangle^A)$
4: $\langle s \rangle^A \leftarrow \sum_i \langle e_i \rangle^A$
5: $\langle r \rangle^A \leftarrow \mathrm{Reciprocal}(\langle s \rangle^A)$
6: $\langle p_i \rangle^A \leftarrow \mathrm{Trunc}(\langle e_i \rangle^A \langle r \rangle^A)$
7: **return** $\langle p \rangle^A$

C. SECURE ATTENTION SOFTMAX

The implemented Softmax accepts a tensor with logical shape $[B, H, L_q, L_k]$ and normalizes along L_k. The current BERT-base test shape is $[2, 12, 128, 128]$, and the implementation supports $L_k \leq 128$. Algorithm 1 gives the protocol structure.

1) Maximum subtraction

For each row, the protocol evaluates a balanced comparison tree. With $L = 128$, the tree has seven levels. Each pairwise maximum uses the framework's signed maximum primitive, whose comparison obtains a sign bit through the Boolean path and returns the selected value as an arithmetic share. Maximum subtraction produces $\delta_i = x_i - m \leq 0$ and avoids overflow in exponentiation. It does not guarantee $\delta_i \geq -\ln 2$, so a separate range-reduction step is required.

2) Range-reduced exponential

The protocol masks values below $-16 \ln 2$ to zero. For active values, it clips the argument to this lower bound and sets $y = \delta/16$, giving $y \in [-\ln 2, 0]$. It then evaluates the cubic polynomial

$$T_3(y) = 1 + y + \frac{y^2}{2} + \frac{y^3}{6} \tag{4}$$

over arithmetic shares and reconstructs the reduced exponential as

$$\widehat{\exp}(\delta) = (T_3(\delta/16))^{16}, \tag{5}$$

using four secure squarings. Every fixed-point product is followed by the configured exact truncation. A final arithmetic mask forces the clipped tail to zero, rather than returning the exponential of the clipping threshold.

The 32-bit ring creates an additional implementation constraint: multiplying two Q12.20 encodings can require more than 32 intermediate bits even when the truncated result is representable. The implementation decomposes each operand into 10-bit low and high limbs. It securely computes the high–high and cross products, truncates the cross term by 10 bits, and combines the terms. This preserves the relevant

Q12.20 product without relying on an unavailable wider ring intermediate.

3) Reduction and reciprocal

Because addition is local over arithmetic shares, each row sum

$$s = \sum_{i=1}^{L} e_i \tag{6}$$

requires no additional communication. For $L \leq 128$, the reciprocal routine securely identifies the highest-bit bucket k for $s \in [2^k, 2^{k+1})$. It forms the piecewise linear seed

$$r_0 = \frac{48}{17} 2^{-(k+1)} - \frac{32}{17} s\, 2^{-2(k+1)} \tag{7}$$

and performs two Newton–Raphson iterations,

$$r_{t+1} = r_t(2 - s r_t). \tag{8}$$

The reciprocal is broadcast within the secret-shared row and multiplied by each approximate exponential. No secret value is opened during the operator.

D. LAYERNORM PROTOCOL CONTRACT

For a hidden vector $x \in \mathbb{R}^d$, LayerNorm computes

$$\mu = \frac{1}{d} \sum_{i=1}^{d} x_i, \tag{9}$$

$$v = \frac{1}{d} \sum_{i=1}^{d} (x_i - \mu)^2, \tag{10}$$

$$y_i = \gamma_i (x_i - \mu)(v + \epsilon)^{-1/2} + \beta_i. \tag{11}$$

Unlike BatchNorm, the statistics are computed per token over the hidden dimension during inference; they cannot be replaced by public running means and variances. The integration contract therefore accepts arithmetic shares of $[B, L, d]$, computes the mean and variance through local reductions and secure multiplications, evaluates a secure reciprocal square root, and applies private or public affine parameters according to the model-owner setting.

The reciprocal-square-root approximation and its normalization interval remain under implementation. The final protocol will report its polynomial or iterative approximation, truncation points, worst-case numerical error over the tested activation range, and communication rounds. Until those details are fixed and validated, this section is a design contract rather than a completed protocol claim.

E. FFN AND RESIDUAL-NORMALIZATION PIPELINE

A BERT FFN transforms each token independently:

$$\mathrm{FFN}(x) = W_2\ \mathrm{GELU}(W_1 x + b_1) + b_2. \tag{12}$$

The two dense products are mapped to CUDA GEMM. Bias addition and residual addition remain in arithmetic sharing. GELU requires an MPC-friendly approximation whose interval must be calibrated against secret activation traces or a conservative public bound. The planned fused pipeline consumes the residual input and FFN output directly, performs the residual addition, and invokes the LayerNorm contract without reconstructing an intermediate tensor.

Fusion here denotes an execution-interface optimization, not a change to BERT semantics. The implementation must preserve the reference order of bias, activation, second projection, residual addition, and normalization. Its benefit will be assessed through tensor-lifetime measurements, host and device memory, communication volume, and end-to-end latency. No fusion speedup is claimed in the present draft.

F. FRAMEWORK INTEGRATION

The initial Transformer entry point is assigned `FUNCTION_IDENTIFIER=200`. The implementation resides in `programs/functions/transformer/softmax_mpc.hpp` and is selected in `protocol_executer.hpp` through the framework's preprocessor dispatch. Future identifiers 201 and 202 are reserved for LayerNorm and the FFN-plus-LayerNorm pipeline, respectively. A complete PIGEON model integration additionally requires a Transformer architecture definition in `nn/PIGEON/architectures/`, layer construction in the FlexNN-facing interface, and model/data serialization compatible with Pygeon.

G. VALIDATION METHODOLOGY

Operator validation compares reconstructed MPC outputs with a PyTorch reference generated from identical seeded tensors. Softmax tests include zero inputs, constant rows, random bounded scores, and rows containing a dominant or strongly negative element. The primary numerical metrics are maximum absolute error, mean absolute error, row-sum error, and the presence of non-finite outputs. End-to-end validation will fix model parameters, samples, and software revision across all parties and will record the maximum completion time among parties, communication, GPU utilization, memory, and model-task accuracy. This separation between operator correctness and system performance prevents throughput results from masking a numerical regression.

V. EXPERIMENTAL EVALUATION

[TODO: Insert the controlled BERT-base experiments, accuracy analysis, ablations, and resource measurements. No result is claimed in the present data-independent draft.]

VI. CONCLUSION

This article targets the systems gap between cryptographically secure Transformer execution and efficient use of modern GPU clusters. The proposed architecture combines mixed-protocol three-party computation with GPU-resident linear algebra and introduces an attention-oriented secure Softmax design. The current implementation establishes the Softmax path and the framework integration point. Completing and evaluating the LayerNorm and FFN pipelines, followed by controlled end-to-end BERT-base measurements, remains necessary before drawing performance or accuracy

conclusions.

• • •

ACKNOWLEDGMENT

[TODO: Add acknowledgments and the IEEE-required disclosure describing any generative-AI assistance used for manuscript drafting.]

REFERENCES

[1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems*, vol. 30, 2017. [Online]. Available: https://proceedings.neurips.cc/paper/7181-attention-is-all-you-need

[2] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proceedings of NAACL-HLT*, 2019, pp. 4171–4186.

[3] C. Juvekar, V. Vaikuntanathan, and A. Chandrakasan, "GAZELLE: A low latency framework for secure neural network inference," in *27th USENIX Security Symposium*, 2018, pp. 1651–1669. [Online]. Available: https://www.usenix.org/conference/usenixsecurity18/presentation/juvekar

[4] P. Mishra, R. Lehmkuhl, A. Srinivasan, W. Zheng, and R. A. Popa, "Delphi: A cryptographic inference service for neural networks," in *29th USENIX Security Symposium*, 2020, pp. 2505–2522. [Online]. Available: https://www.usenix.org/conference/usenixsecurity20/presentation/mishra

[5] P. Mohassel and P. Rindal, "ABY3: A mixed protocol framework for machine learning," in *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security*, 2018, pp. 35–52.

[6] M. Hao, H. Li, H. Chen, P. Xing, G. Xu, and T. Zhang, "Iron: Private inference on transformers," in *Advances in Neural Information Processing Systems*, vol. 35, 2022. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2022/hash/64e2449d74f84e5b1a5c96ba7b3d308e-Abstract-Conference.html

[7] D. Li, H. Wang, R. Shao, H. Guo, E. P. Xing, and H. Zhang, "MPCFormer: Fast, performant and private transformer inference with MPC," in *International Conference on Learning Representations*, 2023. [Online]. Available: https://openreview.net/forum?id=CWmvjOEhgH-

[8] Y. Dong, W. jie Lu, Y. Zheng, H. Wu, D. Zhao, J. Tan, Z. Huang, C. Hong, T. Wei, and W. Chen, "PUMA: Secure inference of LLaMA-7B in five minutes," *arXiv preprint arXiv:2307.12533*, 2023.

[9] H. Wu, W. Fang, Y. Zheng, J. Ma, J. Tan, and L. Wang, "Ditto: Quantization-aware secure inference of transformers upon MPC," in *Proceedings of the 41st International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 235, 2024, pp. 53 346–53 365. [Online]. Available: https://proceedings.mlr.press/v235/wu24d.html

[10] S. Tan, B. Knott, Y. Tian, and D. J. Wu, "CryptGPU: Fast privacy-preserving machine learning on the GPU," in *2021 IEEE Symposium on Security and Privacy*, 2021, pp. 1021–1038.

[11] J.-L. Watson, S. Wagh, and R. A. Popa, "Piranha: A GPU platform for secure computation," in *31st USENIX Security Symposium*, 2022, pp. 827–844. [Online]. Available: https://www.usenix.org/conference/usenixsecurity22/presentation/watson

[12] C. Harth-Kitzerow, A. Suresh, Y. Wang, H. Yalame, G. Carle, and M. Annavaram, "High-throughput secure multiparty computation with an honest majority in various network settings," *Proceedings on Privacy Enhancing Technologies*, vol. 2025, no. 1, pp. 250–272, 2025.

[13] C. Harth-Kitzerow, Y. Wang, R. Rajat, G. Carle, and M. Annavaram, "PIGEON: A high throughput framework for private inference of neural networks using secure multiparty computation," *Proceedings on Privacy Enhancing Technologies*, vol. 2025, no. 3, pp. 88–105, 2025.

[14] C. Harth-Kitzerow, A. Suresh, and G. Carle, "SoK: Truncation untangled: Scaling fixed-point arithmetic for privacy-preserving machine learning to large models and datasets," *Proceedings on Privacy Enhancing Technologies*, vol. 2025, no. 4, pp. 369–391, 2025.